

Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial

Instituto Tecnológico de Buenos Aires



Grupo G136

Integrantes:

- Abraldes, Joaquin (65063)
- Andraca, Mateo (65177)

Fecha de Entrega: 14/06/2026

2026

Tabla de Contenidos

1. Introducción.....	3
2. Modelo Computacional.....	3
2.1 Dominio.....	3
2.2 Lenguaje.....	3
3. Implementación.....	4
3.1 Frontend.....	4
3.2 Backend.....	5
3.3 Adicionales.....	6
3.4 Dificultades Encontradas.....	6
4. Futuras Extensiones.....	6
5. Conclusiones.....	6
6. Bibliografía.....	7

1. Introducción

GymLang es un lenguaje específico de dominio (DSL) que permite describir rutinas de entrenamiento físico y su planificación en el tiempo, y compilarlas a una página HTML interactiva que sirve como guía de la sesión. El compilador está escrito en C sobre la base Flex-Bison-Compiler, usando Flex para el análisis léxico y Bison para el sintáctico.

El proyecto se entregó en tres etapas. En el Stage I se definió el dominio, las construcciones del lenguaje y los casos de prueba. En el Stage II se construyó el frontend (analizador léxico y sintáctico) que transforma un programa en su Árbol de Sintaxis Abstracta (AST). En el Stage III nos enfocamos en el backend, que terminamos dividiendo en dos partes: un análisis semántico que chequea que el AST tenga sentido según las reglas del gimnasio, y la generación del HTML final.

2. Modelo Computacional

2.1 Dominio

El dominio son las rutinas de entrenamiento de gimnasio. Una persona arma rutinas (workout) compuestas por ejercicios de tres modalidades: fuerza (strength), cardiovascular (cardio) e hipertrofia (hypertrophy). Cada modalidad tiene un conjunto propio de atributos relevantes: la fuerza y la hipertrofia se describen con series, repeticiones, peso, descanso, tempo y dropsets, mientras que el cardio se describe con duración, intensidad y distancia.

Los ejercicios pueden agruparse en supersets (varios ejercicios ejecutados de forma consecutiva, una vez por ronda, sin descanso intermedio) y precederse de un bloque de entrada en calor (warmup). Finalmente, las rutinas se organizan en el tiempo mediante una planificación semanal (week, un workout por día) o una planificación recurrente (recurring, los workouts rotan según un patrón de días de entrenamiento y descanso).

El resultado de compilar un programa es una página web que muestra cada rutina por bloques, con un desglose serie por serie, un resumen de volumen total, series totales y tiempo estimado, y un reloj simulado que recorre la planificación para visualizar qué se entrena cada día.

2.2 Lenguaje

La gramática $G = \langle \Sigma, N, \Pi, S \rangle$ queda definida por el frontend: el analizador léxico fija el alfabeto Σ (los tokens) y el sintáctico fija los no terminales N , las producciones Π y el símbolo inicial $S = \text{program}$. Un programa es una secuencia de uno o más workout seguida opcionalmente de un único bloque de planificación.

Reglas principales del lenguaje:

- Un workout tiene un name obligatorio, una description opcional, un warmup opcional y al menos un item (ejercicio o superset).
- Un ejercicio declara su type y luego sus atributos en formato clave=valor;.
- reps y weight aceptan un valor único, una lista separada por comas o rangos con guión (por ejemplo reps=8-12,8-12,6-10). La cantidad de valores de la lista debe coincidir con la cantidad de series.

- tempo se escribe en formato EPCP (tempo=3-1-2-0).
- El dropset es un atributo del último set de los ejercicios de fuerza e hipertrofia, e indica los pesos de cada caída; dropset_reps permite especificar repeticiones distintas por caída.
- La planificación es excluyente: un programa usa week o recurring, no ambos.

Un ejemplo de la sintaxis:

```
workout {
  name=Tren Superior;
  description=push y pull;

  warmup {
    exercise {
      type=cardio;
      name=Cinta;
      duration=5;
      intensity=3;
    }
  }

  exercise {
    type=strength;
    name=Press banca;
    sets=4;
    reps=5;
    weight=100;
    rest=180;
    tempo=3-1-2-0;
  }

  exercise {
    type=strength;
    name=RDL;
    sets=3;
    reps=1-3,5-8,5-8;
    weight=120,100,100;
    rest=120;
  }

  superset {
    rounds=3;
    rest=60;
    exercise {
      type=hypertrophy;
      name=Biceps curl;
      reps=10,10,8;
      weight=20;
    }
    exercise {
      type=hypertrophy;
      name=Fondos;
      reps=12;
    }
  }
}
```

```
}  
  
workout {  
    name=Tren Inferior;  
  
    exercise {  
        type=strength;  
        name=Sentadilla;  
        sets=4;  
        reps=6,6,8,8;  
        weight=100,100,90,90;  
        rest=180;  
    }  
  
    exercise {  
        type=hypertrophy;  
        name=Extension de cuadriceps;  
        sets=3;  
        reps=12-15;  
        weight=50;  
        rest=60;  
    }  
}  
  
week {  
    mon=Tren Superior;  
    tue=Tren Inferior;  
    fri=Tren Superior;  
}
```

Código 2.2.1: Dos rutinas con reps y peso variables por set (listas y rangos) con planificación week.

Sobre los nombres con espacios: como el alfabeto de un nombre es `[A-Za-z][^,;{}=]*`, la coma y la llave están prohibidas dentro de un nombre. Por eso `order=Pecho,Espalda,Piernas`; no es ambiguo aunque los nombres no vayan entre comillas, tal como observó la devolución del Stage I.

3. Implementación

El código fuente se organiza en `src/main/c`, con el frontend en `frontend/` y el backend en `backend/`. El punto de entrada (`EntryPoint.c`) inicializa cada módulo, ejecuta el análisis sintáctico, y si tiene éxito encadena el análisis semántico y la generación de código.

3.1 Frontend

El analizador léxico (`FlexPatterns.l`) usa tres start conditions: `INITIAL` para las palabras clave y delimitadores, `VALUE` para el lado derecho de cada asignación (donde se reconocen enteros, strings, comas y guiones) y `MULTILINE_COMMENT` para los comentarios `/* ... */`. La separación de contextos permite que un nombre de rutina o ejercicio contenga texto libre sin

colisionar con las palabras clave del lenguaje. Un comentario sin cerrar al llegar al fin de archivo se rechaza.

El analizador sintáctico (BisonGrammar.y) es un parser push reentrante. Cada producción tiene asociada una acción semántica (BisonActions.c) que construye el nodo correspondiente del AST, definido en AbstractSyntaxTree.h. El árbol modela el programa completo: listas de workouts, ejercicios con su lista de atributos (un union discriminado por AttributeValueType), supersets, warmups y los dos tipos de planificación. Los %destructor de la gramática liberan los nodos ante errores de sintaxis, evitando fugas en los programas rechazados.

3.2 Backend

El backend se divide en dos fases.

Análisis semántico (backend/semantic-analysis/). Como el frontend es libre de contexto, deja pasar cosas que sintácticamente están bien pero no tienen sentido en el dominio, como un ejercicio de fuerza sin reps. El análisis semántico recorre el AST y filtra esos casos, usando dos componentes:

- **Tabla de símbolos** (SymbolTable.c) que registra los nombres de los workouts. Detecta nombres duplicados al definirlos y resuelve las referencias de las planificaciones week y recurring contra las definiciones existentes.
- **Analizador** (SemanticAnalyzer.c) que valida cada ejercicio según un sistema de tipos y el scope en el que aparece. El scope (SCOPE_WORKOUT, SCOPE_WARMUP, SCOPE_SUPERSET) cambia qué atributos son obligatorios: un ejercicio de fuerza suelto exige sets y reps, pero dentro de un superset las sets son implícitas (una por ronda), por lo que la longitud de las listas de reps y weight se valida contra la cantidad de rondas en lugar de contra sets.

Las validaciones que aplica son: atributos permitidos por tipo (un cardio no puede tener weight, un strength no puede tener distance), atributos obligatorios por tipo y scope, atributos repetidos, rangos de valores (sets ≥ 1 , duration ≥ 1 , intensity entre 1 y 10, repeticiones positivas, rangos bien formados), consistencia entre la longitud de las listas y la cantidad de series/rondas, coincidencia entre la cantidad de pesos del dropset y de dropset_reps, unicidad de los días en week, y existencia de los workouts referenciados en las planificaciones. Cuando encuentra un error lo reporta y la compilación termina con estado de fallo.

Generación de código (backend/code-generation/Generator.c). Recorre el AST validado y emite, mediante una función output que escribe sobre un archivo (output.html), una página HTML autocontenida. Cada nodo del árbol se traduce localmente: el program arma el documento y la sección de planificación, cada workout produce una tarjeta con sus bloques, y cada exercise produce su detalle de atributos y el desglose serie por serie. Durante el recorrido se calcula el resumen de la sesión (volumen en kg, series totales y tiempo estimado); para las listas y rangos se toma el valor representativo de cada serie, y el tiempo se estima a partir del tempo, las repeticiones y el descanso.

3.3 Adicionales

El HTML generado es interactivo. Incluye casillas de progreso por serie, temporizadores de descanso que arrancan con un clic, y un reloj simulado con un control de velocidad (minutos simulados por segundo real). El reloj recorre la planificación y resalta el día activo, de modo que una rutina de una semana o un ciclo recurrente se puede observar de forma acelerada sin esperar tiempo real. Esto responde a la sugerencia de la devolución del Stage I de incluir un multiplicador para depurar la animación temporal.

No hace falta ningún runtime especial: alcanza con abrir el HTML en un navegador. Es un único archivo con el CSS y el JavaScript embebidos, sin dependencias ni servidor, así que se puede mover a cualquier lado (y como no carga recursos externos, tampoco hay problemas de Same Origin).

3.4 Dificultades Encontradas

- **Scopes y atributos obligatorios.** Los supersets nos complicaron más de lo que esperábamos. Como dentro de un superset las series son implícitas, no podíamos validar todos los ejercicios con las mismas reglas. Al principio intentamos resolverlo en el parser, pero terminamos pasándole el scope al validador para que supiera en qué contexto estaba cada ejercicio.
- **Listas variables por serie.** reps y weight admiten valor único, lista o rangos, lo que complica tanto la validación de longitud como el cálculo del volumen. Se unificó con una función que devuelve el valor representativo de cada serie y replica el valor único cuando corresponde.
- **Salida sin fugas de memoria.** Como compilamos con AddressSanitizer, cualquier fuga nos rompía los tests, así que tuvimos que ser cuidadosos liberando todo. La tabla de símbolos y el AST se liberan enteros, y para evitar buffers intermedios la generación escribe directo sobre el archivo de salida.

4. Futuras Extensiones

De cara al futuro, hay varias líneas por las que el proyecto podría seguir creciendo. Una de las más interesantes sería permitir reutilizar definiciones de ejercicios entre distintas rutinas, armando una especie de catálogo de ejercicios con nombre que se resuelva a través de la tabla de símbolos, para no tener que repetir las mismas definiciones una y otra vez. También nos gustaría dar soporte a planificaciones más amplias, como mesociclos que abarquen varias semanas, y no quedarnos solo con esquemas más acotados. Por último, pensamos en ampliar los formatos de salida más allá del HTML actual, de manera que se pueda generar, por ejemplo, un calendario en formato .ics o directamente un PDF según lo que cada uno necesite.

5. Conclusiones

El proyecto recorre las fases clásicas de un compilador sobre un dominio acotado y concreto. El frontend genera el AST, el análisis semántico restringe el superconjunto que acepta la gramática a los programas que tienen sentido en el dominio (apoyado en una tabla de símbolos, un sistema de tipos por modalidad de ejercicio y la noción de scope), y la generación de código produce un artefacto útil y portable. La suite de pruebas (18 casos de aceptación y 22

de rechazo) cubre tanto los errores sintácticos como los semánticos, y todos pasan dentro del entorno reproducible de Docker.

6. Bibliografía

1.  (2026-03-12) Proyecto Especial
2.  (2026-03-12) Modelo Computacional
3.  (2024-09-25, v0.2.0) Análisis Léxico
4.  (2024-05-08, v0.1.0) Análisis Sintáctico
5.  (2024-05-08, v0.1.0) Análisis Sintáctico
6.  (2024-09-25, v0.2.0) Análisis Semántico
7.  (2024-05-16, v0.1.0) Generación de Código